Plague Containment

Nelson David Molina Ramos
Computer Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Brayan Estiven Aguirre Aristizabal
Computer Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

Juan Sneyder Méndez Gil
Computer Engineering Program
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia

***Abstract*—This paper describes the design of Plague Containment, an interactive grid-based strategy game developed as part of the Computer Sciences I course project. The game models a city as an 8×8 grid of districts, each with a different infection risk level. The player must stop a spreading plague using two algorithmic tools: a greedy vaccination strategy, which always targets the highest-risk uninfected district bordering the infection, and a backtracking quarantine planner, which looks for a ring of districts that encloses the infected zone without cutting off any healthy part of the city. This paper presents the problem background, the city model, the design of both algorithms through pseudo-code, the JSON communication schema that bridges the C++ engine with the Python algorithms and the Pygame interface, and a wireframe of the game screen.**

***Index Terms*—greedy algorithm, backtracking, epidemic containment, grid model, game design, algorithm design, system integration, JSON bridge.**

I. Introduction

Plague Containment is a turn-based strategy game that puts the player in charge of stopping an outbreak across a city grid. Each district has an infection risk level, and the plague spreads every turn to neighbor districts. The player has two tools: vaccinate a district to make it immune, or establish a quarantine perimeter to isolate the infected zone.

Two algorithms assist the player. The greedy advisor scans the border of the infected zone and recommends the healthy district with the highest-risk to vaccinate. The backtracking algorithm searches for a valid quarantine ring that surrounds all infected districts while ensuring no healthy district becomes cut off from the rest of the city.

The goal of this paper is to describe the city model, present the design and pseudo-code of both algorithms, define the JSON schema that connects the three system layers, and show the wireframe of the game interface.

II. City Grid Model

A. The Grid

The city is represented as an 8×8 grid of districts. Each cell in the grid corresponds to one district. Two districts are considered neighbors if they share a side (up, down, left, right).

B. District States

Every district is always in exactly one of four possible states:

- **Healthy** — not yet affected by the plague.
- **Infected** — currently carrying the plague and spreading it to neighbors.
- **Vaccinated** — immunized by the player; cannot become infected.
- **Quarantined** — placed under lockdown by the backtracking planner to form the containment perimeter.

C. Risk Level

Each district has a risk level, which is a whole number between 1 and 10 assigned at the start of the game. A higher risk level means that the district is more vulnerable to contracting the plague (for example, due to high population density or poor sanitation). The risk level does not change during the game.

D. How the Plague Spreads

At the end of each turn, the plague attempts to spread. For every infected district, each of its healthy neighbors has a chance of becoming infected. Districts that are vaccinated or quarantined block the spread entirely.

Figure 1 shows an example of the grid mid-game.

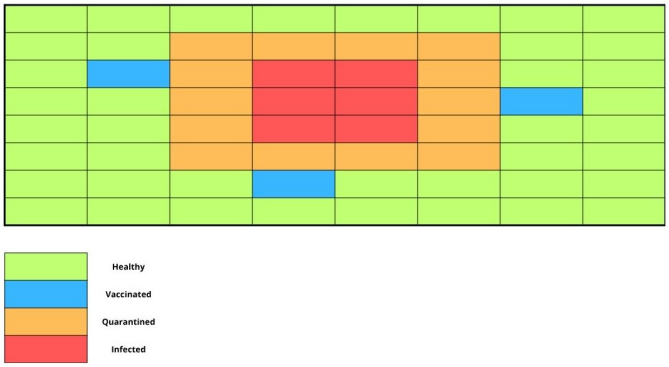

Fig. 1. Example of the grid mid-game

III. Greedy Vaccination Algorithm

A. What It Does

The greedy algorithm helps the player decide which district to vaccinate on each turn. It works by looking at every healthy district that directly borders an infected one, and recommending the one with the highest risk level. The idea is simple: vaccinate the most dangerous unprotected neighbor before the plague reaches it.

B. Why Greedy Works Here

Greedy algorithms are a good fit when each decision can be made locally without looking far ahead. In this case, the most urgent district to protect is always the one most likely to spread the plague further — which is the one with the highest risk. The greedy choice aligns directly with the immediate threat.

C. Pseudo-code

Algorithm 1 Greedy Vaccination

Input: The game grid with all district states and risk levels
Output: The best district to vaccinate this turn, or None

```
candidates ← empty list
for each infected district d on the grid do
    for each neighbor n of d do
        if n is HEALTHY then
            add n to candidates (avoid duplicates)
        end if
    end for
end for
if candidates is empty then
    return NONE
end if
best ← first district in candidates
for each district c in candidates do
    if c.risk > best.risk then
        best ← c
    end if
end for
mark best as VACCINATED
return best
```

IV. Backtracking Quarantine Algorithm

A. What It Does

The backtracking algorithm finds a set of healthy districts to quarantine so that the infected zone is completely enclosed. The result must satisfy two conditions at the same time:

1) **Full enclosure.** Every infected district must be completely surrounded — none of its neighbors can remain a free healthy district.
2) **No isolation.** After quarantine is placed, every healthy district that is not quarantined must still be reachable from the rest of the healthy city. No healthy area can become a disconnected island.

B. Why Backtracking

This is not a greedy problem because a locally good choice can create a globally bad outcome. For instance, quarantining a district that seems necessary might accidentally cut off a healthy neighborhood. Backtracking allows the algorithm to try a choice, check if both conditions are met, and undo that choice if they are not — exploring alternatives until a valid solution is found.

C. Pseudocode

Algorithm 2 is the starting point. Algorithm 3 is the recursive function it calls.

Algorithm 2 Backtracking Quarantine — Entry Point

Input: The game grid with all district states
Output: A valid quarantine set, or NONE if impossible

```
border ← all healthy neighbors of infected districts
quarantine ← empty set
success ← SEARCH(border, index 0, quarantine)
if success then
    return quarantine
else
    return NONE
end if
```

Algorithm 3 Search — Recursive Backtracking Step

Input: Border district list, current index i, partial quarantine set *quarantine*
Output: TRUE if a complete valid quarantine was found, FALSE otherwise

```
if infected zone is fully enclosed by quarantine then
    return TRUE {Both conditions satisfied — solution found}
end if
if no more border districts to try then
    return FALSE {Ran out of candidates without full enclosure}
end if
d ← border[i]
{— Option A: quarantine district d —}
add d to quarantine
temporarily mark d as QUARANTINED on the grid
if remaining healthy districts are still all connected then
    if SEARCH(border, i + 1, quarantine) then
        return TRUE
    end if
end if
{— Backtrack: undo quarantine of d —}
remove d from quarantine
restore d to HEALTHY on the grid
{— Option B: skip district d —}
if SEARCH(border, i + 1, quarantine) then
    return TRUE
end if
return FALSE
```

V. How the Two Algorithms Work Together

The greedy and backtracking algorithms serve different roles during a game turn. Figure 2 shows the sequence of events within a single turn.

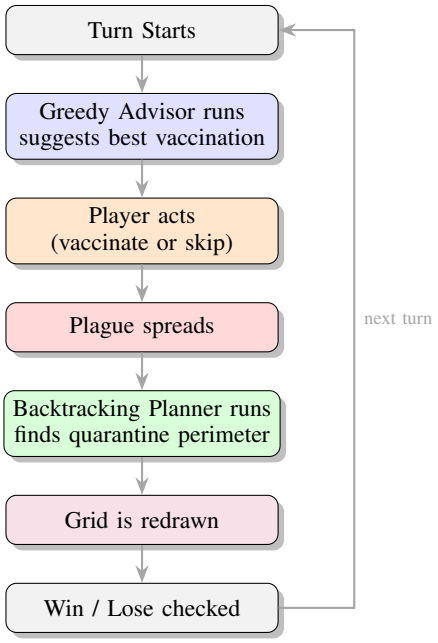

Fig. 2. Sequence of events inside one game turn.

The greedy algorithm runs quickly and gives the player a recommendation *before* they act. The backtracking planner runs *after* the plague spreads so that the displayed quarantine perimeter always reflects the current state of the grid.

VI. System Architecture and Integration

The Plague Containment game is built on a three-layer architecture in which each layer is implemented in a different language. The C++ engine manages the core data structures (the infection linked list and the district AVL tree). The Python layer implements the greedy and backtracking algorithms. The Pygame module renders the graphical interface and captures user input. These three layers do not call each other directly; instead, they communicate through a shared JSON file that acts as a language-neutral bridge.

A. Why a JSON Bridge?

C++ and Python are compiled and interpreted languages, respectively, and mixing them via direct bindings (e.g. `pybind11`) adds complexity beyond the scope of this course. A JSON file provides a simple, human-readable interface that both languages can read and write using their standard libraries (`nlohmann/json` in C++, `json` in Python). This also makes debugging straightforward: at any point during development, a team member can open the JSON file and inspect the full game state.

B. Component Responsibilities

Table I summarizes what each component owns.

TABLE I
Responsibility of each system component.

Layer	Language	Responsibilities
Engine	C++	Maintain the infection chain (linked list) and district risk index (AVL tree). Update plague spread each turn. Write the updated game state to JSON.
Algorithms	Python	Read the game state from JSON. Run the greedy advisor and the backtracking planner. Write algorithm results (recommendation, quarantine set) back to JSON.
Interface	Python / Pygame	Read the game state and algorithm results from JSON. Render the grid, highlights, and HUD. Capture player input and write the chosen action to JSON.

C. Communication Flow

Figure 3 illustrates how data moves through the system during a single game turn.

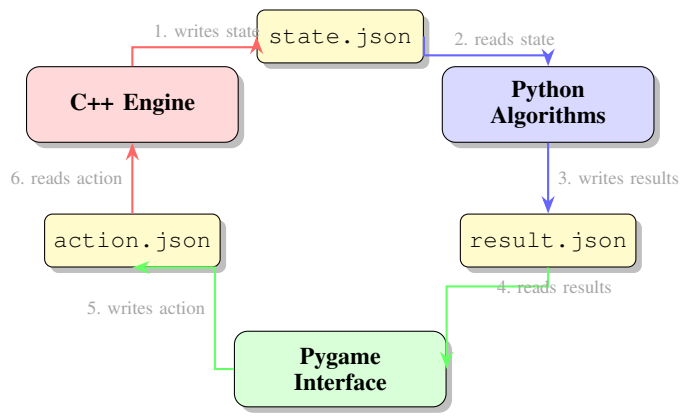

Fig. 3. Data flow between the three system components through JSON files. Numbers indicate the order within one turn.

The cycle works as follows:

1) The **C++ engine** updates the game state (plague spread, data structures) and writes the result to `state.json`.
2) The **Python algorithm module** reads `state.json`, runs the greedy advisor and the backtracking planner, and writes its recommendations to `result.json`.
3) The **Pygame interface** reads both files, renders the grid with the algorithm suggestions highlighted, and waits for the player to act.
4) When the player clicks, the interface writes the chosen action to `action.json`.
5) The **C++ engine** reads `action.json`, applies the action (e.g. vaccinate a district), advances the turn, and the cycle repeats.

VII. JSON Schema Design

Three JSON files mediate all inter-component communication. This section defines the structure of each one.

A. Game State (`state.json`)

This file is written by the C++ engine and read by both Python and Pygame. It contains the complete snapshot of the game at the start of a turn.

Listing 1. Example `state.json` (turn 3).

```
{
  "turn": 3,
  "grid_size": 8,
  "districts": [
    {
      "row": 0,"col": 0,
      "risk": 5,
      "state": "healthy"
    },
    {
      "row": 0,"col": 1,
      "risk": 8,
      "state": "infected"
    },
    {
      "row": 1,"col": 0,
      "risk": 3,
      "state": "vaccinated"
    },
    {
      "row": 1,"col": 1,
      "risk": 7,
      "state": "quarantined"
    }
  ],
  "infection_chain": [
    {"row": 3,"col": 4,"turn_infected": 1},
    {"row": 3,"col": 5,"turn_infected": 1},
    {"row": 2,"col": 4,"turn_infected": 2},
    {"row": 4,"col": 4,"turn_infected": 3}
  ],
  "stats": {
    "total_infected": 4,
    "total_vaccinated": 6,
    "total_quarantined": 2,
    "total_healthy": 52
  }
}
```

Key design decisions:

- Each district is identified by its `(row, col)` pair, which maps directly to a grid cell.
- The `state` field uses one of four string values: `"healthy"`, `"infected"`, `"vaccinated"`, or `"quarantined"`.
- The `infection_chain` array mirrors the linked list maintained by C++. It preserves insertion order so that Python and Pygame can reconstruct the chronological spread of the plague.
- The `stats` object provides precomputed counters so that the interface can display them without iterating over the full grid.

B. Algorithm Results (`result.json`)

This file is written by the Python algorithm module after processing the game state. It contains the greedy recommendation and the backtracking quarantine plan.

Listing 2. Example `result.json`.

```
{
  "greedy_recommendation": {
    "row": 2,"col": 5,
    "risk": 9,
    "reason": "highest-risk healthy neighbor"
  },
  "quarantine_plan": {
    "found": true,
    "districts": [
      {"row": 2,"col": 3},
      {"row": 4,"col": 5},
      {"row": 5,"col": 4},
      {"row": 3,"col": 6}
    ],
    "cost": 4
  }
}
```

If the backtracking algorithm cannot find a valid quarantine, the `found` field is set to `false` and the `districts` array is empty.

C. Player Action (`action.json`)

This file is written by the Pygame interface when the player makes a decision and is read by the C++ engine to apply the change.

Listing 3. Example `action.json`.

```
{
  "turn": 3,
  "action_type": "vaccinate",
  "target": {"row": 2,"col": 5}
}
```

The `action_type` field can be `"vaccinate"` (the player immunizes a district), `"quarantine"` (the player accepts the backtracking plan), or `"skip"` (the player passes the turn without acting).

VIII. Pygame Interface Design

A. Wireframe Overview

The game window is divided into two regions: the main grid area on the left and a side panel on the right. Figure 4 shows the planned layout.

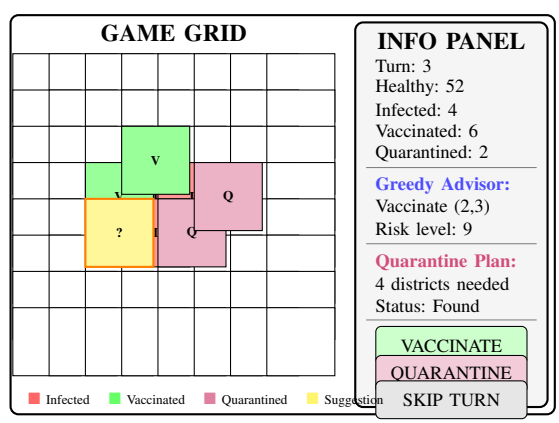

Fig. 4. Wireframe of the Pygame interface showing the grid, info panel, and action buttons.

B. Visual Encoding

Each district state is represented by a distinct color on the grid:

- **White** — healthy district.
- **Red** — infected district.
- **Green** — vaccinated district.

- **Purple** — quarantined district.
- **Yellow border** — district recommended by the greedy advisor.

Each cell also displays its risk level as a number (1–10) in the center, so the player can make informed decisions.

C. Player Interaction

The player interacts with the game through three mechanisms:

1) **Click on a grid cell** to select a district for vaccination. The selected cell is highlighted, and the side panel updates to show its risk level.
2) **Click the VACCINATE button** to confirm vaccinating the selected district.
3) **Click the QUARANTINE button** to accept the backtracking plan and quarantine all districts in the proposed set.
4) **Click SKIP TURN** to pass without acting.

After any action, the interface writes `action.json` and signals the C++ engine to process the next turn.

D. Detailed Wireframe

Figure 5 shows a more detailed wireframe that includes the color legend, risk numbers inside each cell, and the side-panel layout used during gameplay.

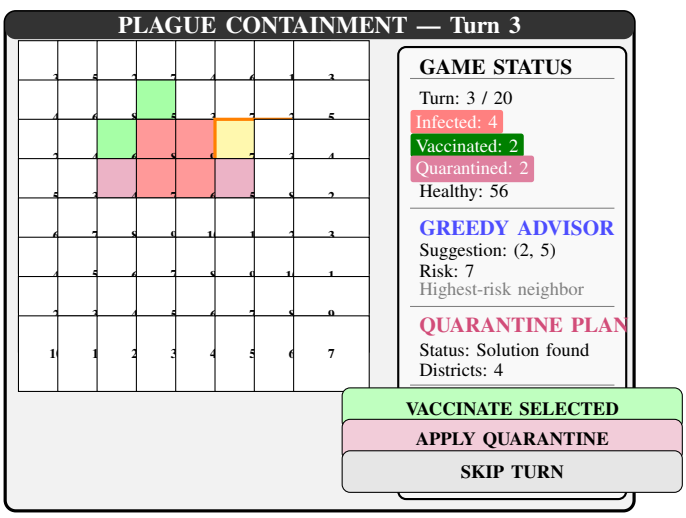

Fig. 5. Detailed wireframe of the game interface. Each cell shows its risk level. Colors encode district states. The yellow-bordered cell marks the greedy suggestion.

IX. Conclusions

This paper has presented the problem background and the complete design for *Plague Containment*. The city is modelled as an 8×8 grid where each district carries a risk level and can be in one of four states. Two algorithms drive the game mechanics: a greedy vaccination advisor that always targets the highest-risk unprotected border district, and a backtracking quarantine planner that exhaustively searches for a valid containment perimeter while respecting the connectivity of the healthy city.

The system is organized in three layers — a C++ engine for data structures, a Python module for algorithms, and a Pygame front-end for rendering and input — connected through three JSON files (`state.json`, `result.json`, and `action.json`) that form a language-neutral communication bridge.

The next steps for the project are to implement both algorithms in Python, build the Pygame interface, and develop the C++ engine that manages the game state.

X. Work Distribution

Table II describes the responsibility of each team member during the Checkpoint phase. Each member contributed one major component of the game design.

TABLE II
Work distribution for the Checkpoint deliverable.

Team Member	Role	Deliverables
Nelson David Molina Ramos	Data Structures (C++)	Design and diagrams of the infection linked list and the district-risk AVL tree. Explanation of how each structure supports the game engine.
Brayan Estiven Aguirre Aristizabal	Algorithms (Python)	Pseudo-code and step-by-step explanation of the Greedy Vaccination algorithm and the Backtracking Quarantine algorithm.
Juan Sneyder Méndez Gil	Interface & Integration	Pygame wireframe design, JSON schema specification (`state.json`, `result.json`, `action.json`), and system architecture describing how the C++ engine, Python algorithms, and Pygame interface communicate.

All three components were designed to be consistent with each other: the JSON fields map directly to the data structures defined by the engine developer, the algorithm outputs match the fields read by the interface, and the wireframe reflects the information provided by both the engine and the algorithms.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

[3] R. Pastor-Satorras, C. Castellano, P. Van Mieghem, and A. Vespignani, "Epidemic processes in complex networks," *Reviews of Modern Physics*, vol. 87, no. 3, pp. 925–979, 2015.